Proyecto 2 - Informe de la Fase 2

Agencia Metropolitana de Transporte: ¿quién deja de usar la red? Universidad Rafael Landívar, Ciencia de Datos, segundo semestre 2026. Daniel Paz (1200022) y Jonathan García (1045222).

Los datos son simulados y no corresponden a ningún operador ni usuario real. La Fase 1 (datos, etiqueta, features y modelo base) está en informe_fase1.pdf; aquí se resume lo necesario para leer esta fase.

Recordatorio. La unidad es la persona. La etiqueta es 1 si la persona no tiene ningún abordaje válido en los 30 días que empiezan en el corte. Corte A (1 de julio): 7,095 / 59,491 = 0.1193. Corte B (31 de julio): 6,387 / 58,113 = 0.1099. La métrica para comparar modelos es la PR-AUC, con 5 folds estratificados del corte A (semilla 42), los mismos para todos los modelos.

2.1 Ensemble y sesgo-varianza

Entrenamos dos ensembles con los mismos 5 folds de la Fase 1: HistGradientBoostingClassifier (boosting, learning_rate 0.05, 300 árboles de hasta 31 hojas, mínimo 100 personas por hoja, l2 = 1.0) y Random Forest (bagging, 300 árboles, mínimo 20 personas por hoja). Para los árboles las variables no se escalan; el perfil se codifica con one-hot dentro del Pipeline.

Modelo	PR-AUC entrenamiento	PR-AUC validación (media ± desv.)	ROC-AUC validación
Baseline 1: tasa del corte A	0.119	0.119 ± 0.000	0.500
Baseline 2: días desde el último viaje	0.402	0.403 ± 0.014	0.826
Baseline 3: menos viajes en 28 días	0.254	0.254 ± 0.004	0.743
Logística L2, λ = 100	0.416	0.415 ± 0.016	0.831
Logística L1, λ = 10	0.415	0.414 ± 0.016	0.831
Logística Elastic Net, λ = 10	0.415	0.415 ± 0.016	0.831
HistGradientBoosting	0.544	**0.426 ± 0.015**	0.841
Random Forest	0.533	0.424 ± 0.014	0.841

Sesgo-varianza, modelo por modelo:

- Baselines: sesgo alto. No aprenden nada (una constante o una sola variable), así que entrenamiento y validación son iguales y bajos.
- Logísticas: sesgo alto. Entrenamiento y validación casi idénticos (0.416 contra 0.415): no se sobreajustan, pero una frontera lineal no capta más de lo que ya da la recencia. Castigar menos no ayuda, como mostró la curva de λ.
- HistGradientBoosting: varianza moderada. Rinde 0.118 más en entrenamiento que en validación, pero su validación es la mejor de todas: está más cerca del equilibrio que la logística, con algo de sobreajuste controlado por el mínimo de 100 personas por hoja.

- Random Forest: varianza moderada, igual que el boosting (brecha de 0.109), y con validación apenas por debajo.

Importancia por permutación, calculada en el fold de validación (no en entrenamiento) con PR-AUC como métrica, para el boosting y promediada en los 5 folds:

#	Feature	Caída de PR-AUC al permutar	Desv. entre folds
1	dias_desde_ultimo_viaje	0.2457	0.0147
2	perfil	0.0152	0.0056
3	tendencia_14d	0.0103	0.0035
4	viajes_14d	0.0090	0.0049
5	viajes_7d	0.0084	0.0066
6	dias_activos_28d	0.0073	0.0051
7	viajes_28d	0.0066	0.0032
8	n_zonas	0.0051	0.0033
9	gasto_28d	0.0045	0.0029
10	prop_hora_pico	0.0033	0.0019

La recencia domina: sin ella el modelo pierde 0.25 de PR-AUC; las demás aportan poco cada una. Las proporciones por operador y n_operadores no aportan nada (importancia cero o negativa).

Decisión: el boosting va a producción. Le gana a la logística en los 5 folds, no solo en promedio (de +0.007 a +0.016 de PR-AUC), y con el mejor umbral de cada uno deja Q11,480 contra Q11,015 en el corte A. La mejora es modesta, pero el costo de la complejidad también lo es: entrena en 2 segundos, se guarda como un solo archivo con el preprocesamiento adentro, y la importancia por permutación nos deja explicar qué usa. Si la Agencia necesitara explicar cada contacto con un coeficiente, la logística sería la alternativa razonable a cambio de menos ganancia.

2.2 Umbral según costo y test único

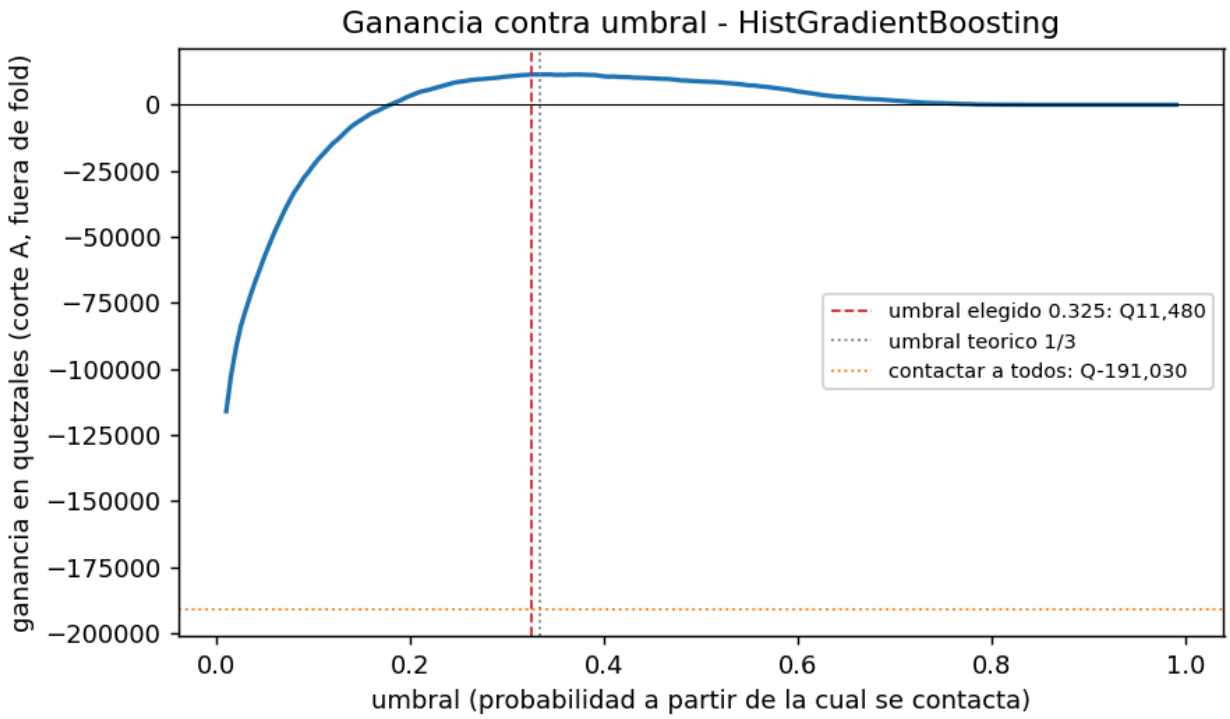
Ganancia. Contactar cuesta Q5. Si la persona se iba a ir, la campaña la retiene en el 25% de los casos y una persona retenida vale Q60:

- Contactar a quien se iba: $0.25 \times Q60 - Q5 = Q15 - Q5 =$ **+Q10**
- Contactar a quien no se iba: **−Q5**
- Ganancia de contactar a un grupo: $\mathbf{G = 10 \times VP - 5 \times FP}$

Si las probabilidades estuvieran bien calibradas, conviene contactar a quien cumpla $p \times 15 - 5 > 0$, es decir $p > 5 / 15 =$ **1/3**.

Umbral elegido: 0.325, el que maximiza G sobre las predicciones fuera de fold del corte A (barrido de 0.01 a 0.99 cada 0.005). Queda casi en el 1/3 teórico, señal de que las probabilidades del boosting están razonablemente calibradas.

En el corte A (fuera de fold)	Umbral 0.325	Umbral 1/3	Contactar a todos
Contactados	6,320	6,009	59,491
VP / FP	2,872 / 3,448	2,767 / 3,242	7,095 / 52,396
Precisión	0.454	0.460	0.119
Recall	0.405	0.390	1.000
Ganancia	**Q11,480** = 10 × 2,872 − 5 × 3,448	Q11,460	−Q191,030



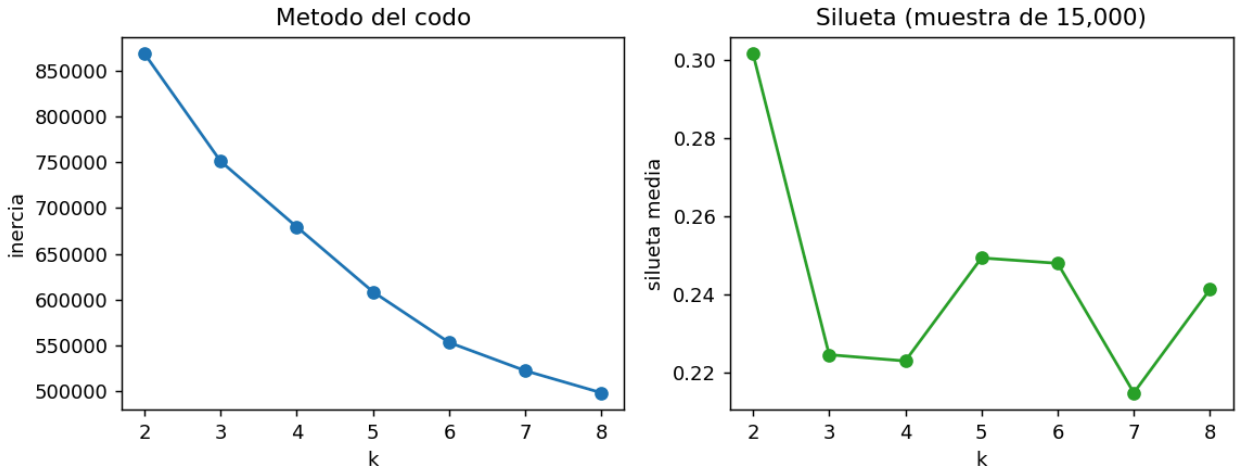
Test único en el corte B. El 3 de noviembre de 2026 a las 01:44, en el commit 3a6bb7c, entrenamos el boosting con todo el corte A, lo guardamos como modelos/modelo_abandono_v1.joblib y lo evaluamos una sola vez en el corte B (python -m ml.evaluar_test_b, que se niega a correr si la evaluación ya existe o si hay cambios sin commit).

Corte B	Valor
PR-AUC del modelo	**0.386**
Baseline 1 (tasa)	0.110
Baseline 2 (días sin viajar)	0.359
ROC-AUC	0.839
Matriz de confusión, umbral 0.325	VP 2,746 · FP 3,982 · FN 3,641 · VN 47,744
Precisión / recall	0.408 / 0.430
Ganancia	**Q7,550** = 10 × 2,746 − 5 × 3,982

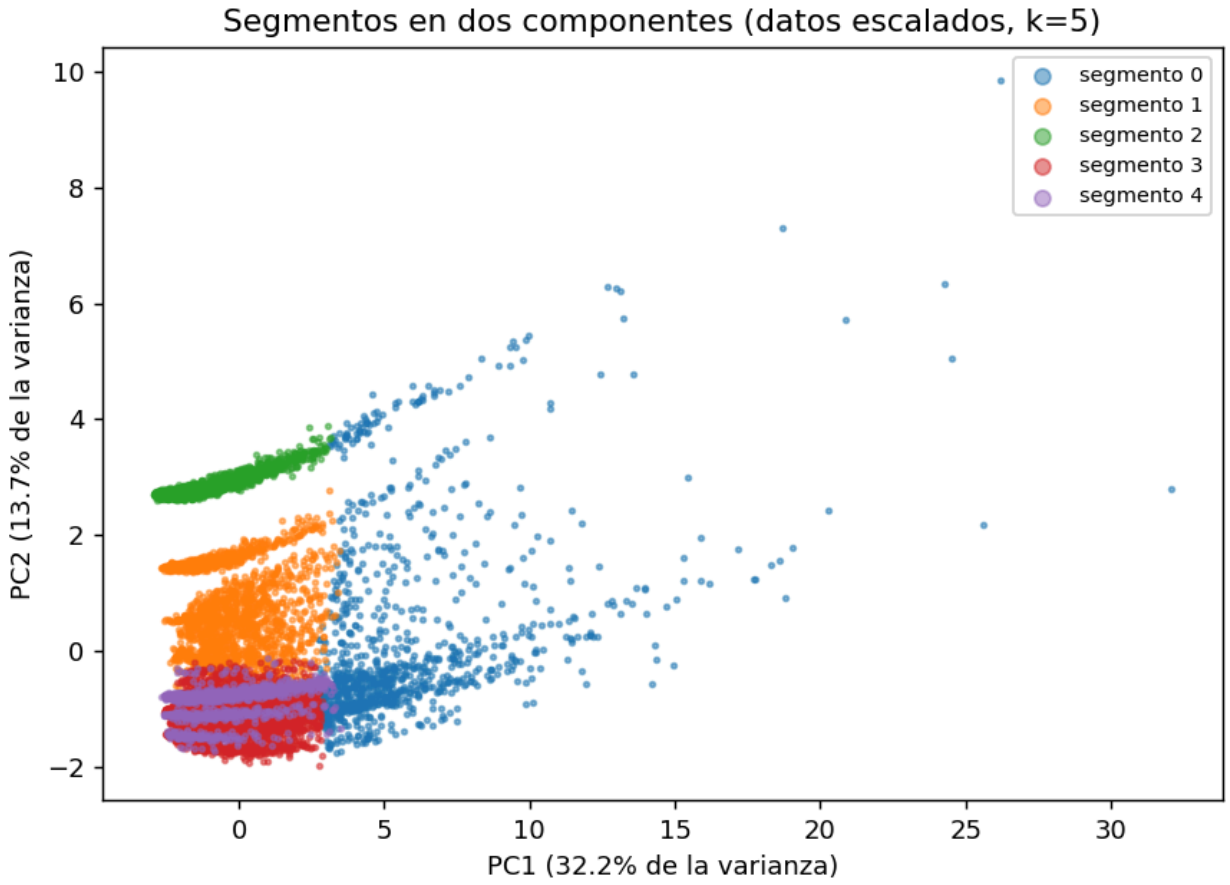
Sale peor que en validación (0.426 y Q11,480) y no ajustamos nada después. La razón principal es que en B abandona menos gente: 11.0% contra 11.9% en A. La PR-AUC y la ganancia dependen de cuántos positivos hay: con menos personas por irse, cada contacto acierta menos aunque el orden sea igual de bueno. La ROC-AUC, que no depende de la tasa, casi no cambia (0.841 en validación, 0.839 en B), y el baseline de recencia cae en la misma proporción (de 0.403 a 0.359). Contra ese baseline el modelo gana incluso más en B (+0.027) que en validación (+0.023).

2.3 Segmentación con KMeans y PCA

KMeans sobre las 18 features numéricas del corte A, escaladas con StandardScaler (el perfil no entra porque es categórico). Probamos k de 2 a 8. La silueta más alta es k = 2 (0.302), pero solo separa a los viajeros muy frecuentes del resto, y eso no sirve para decidir campañas. El siguiente pico de silueta es **k = 5** (0.249), justo donde el codo empieza a aplanarse, así que elegimos 5.



La proyección en dos componentes principales explica el 45.9% de la varianza (PC1 32.2%, PC2 13.7%):



Seg.	Nombre	Tamaño	Viajes 28d	Días activos	Días desde último viaje	Operador principal	Tasa de abandono
0	Viajeros intensivos	5,461	53	21	0.3	Transurbano (70%) y otro	0.019
1	Usuarios de MetroRiel	10,082	9	7	2.6	MetroRiel (80%)	0.125
2	Usuarios de Aerómetro	10,028	6	5	3.7	Aerómetro (100%)	0.145
3	Usuarios de Transurbano	25,482	9	7	2.6	Transurbano (100%)	0.118
4	Usuarios de Transmetro	8,438	9	7	2.6	Transmetro (100%)	0.149

Valores en medianas. La tasa de abandono por segmento es descriptiva: dice dónde se concentra el abandono, no que el segmento lo cause. El segmento no se usa como feature del clasificador.

Acciones de retención:

- Usuarios de Transmetro (14.9%, la tasa más alta): recarga de cortesía condicionada a volver a viajar en la semana siguiente, junto con un aviso de frecuencias en sus líneas.

- Usuarios de Aerómetro (14.5%): un transbordo sin costo hacia Transmetro o Transurbano, para que el teleférico deje de ser un viaje aislado. Ojo: es también el segmento donde no podemos saber si la persona sigue viajando con otra tarjeta.
- Usuarios de MetroRiel (12.5%): pase semanal con descuento, porque viajan en días hábiles y en hora pico.
- Viajeros intensivos (1.9%): no gastar presupuesto de la campaña en ellos; basta con un reconocimiento sin costo.

2.4 MLOps: predecir, guardar y documentar

- **predecir.py**: `python predecir.py --datos datos_red --fecha-corte 2026-08-30`. Carga los datos de la carpeta con el pipeline desde cero, calcula las features del corte, carga el modelo y escribe predicciones/predicciones_2026-08-30.csv con unidad_id, tarjetas (las llaves originales separadas por ";"), puntaje y contactar. No reentrena.
- **Modelo guardado**: modelos/modelo_abandono_v1.joblib, con el Pipeline completo (preprocesamiento y modelo), el umbral, la lista de columnas, el commit y la versión de scikit-learn.
- **Entorno reproducible**: versiones fijas en requirements.txt, semilla 42 y los pasos en el README.
- **Predicciones del corte C**: 57,701 personas, de las cuales 7,872 superan el umbral y se recomienda contactar.
- **Ficha del modelo**: ficha_modelo.pdf.

Prueba de invariancia, ensayada. No tenemos septiembre, pero el corte B permite hacer la misma prueba: predijimos B con los archivos crudos recortados al 31 de julio y con los datos completos, que traen 30 días más. Los dos CSV salieron idénticos byte por byte (58,113 filas, 0 distintas; python -m ml.prueba_invariancia).

En ese ensayo encontramos un problema real: dos corridas de predecir.py sobre los mismos datos daban 2,759 puntajes distintos. La causa era que DuckDB suma en paralelo y la suma de tarifas decimales (por ejemplo Q1.30) cambiaba en el último decimal de máquina según el orden de los hilos; el boosting corta las variables en intervalos y esa diferencia mínima movía a algunas personas de intervalo. Lo corregimos sumando en centavos enteros. No reentrenamos el modelo ni repetimos el test en B: es el mismo dato con la aritmética estabilizada. Desde ese cambio, tres corridas seguidas dan el mismo CSV.

Métricas del proyecto

Categoría	Resultado
Problema	A: 7,095 / 59,491 = 0.1193 · B: 6,387 / 58,113 = 0.1099 · C: 57,701 personas
Validación	PR-AUC 0.426 ± 0.015 (boosting) contra 0.403 ± 0.014 (baseline de recencia) y 0.119 (tasa)
Decisión	Umbral 0.325, precisión 0.454, recall 0.405, ganancia Q11,480 en A
Test	Corte B, 3 nov 2026, commit 3a6bb7c: PR-AUC 0.386 (baselines 0.110 y 0.359), Q7,550
Fuga	Prueba de recorte: 0 diferencias en Silver y en crudo. Invariancia ensayada con B: idéntica

Categoría	Resultado
Rendimiento	Carga desde cero 25 a 67 s según la máquina · features de 3 cortes 4 a 10 s · validación cruzada del boosting 6 s · entrenamiento final 2 s · predecir.py 38 s con el pipeline incluido (4 s sin él)